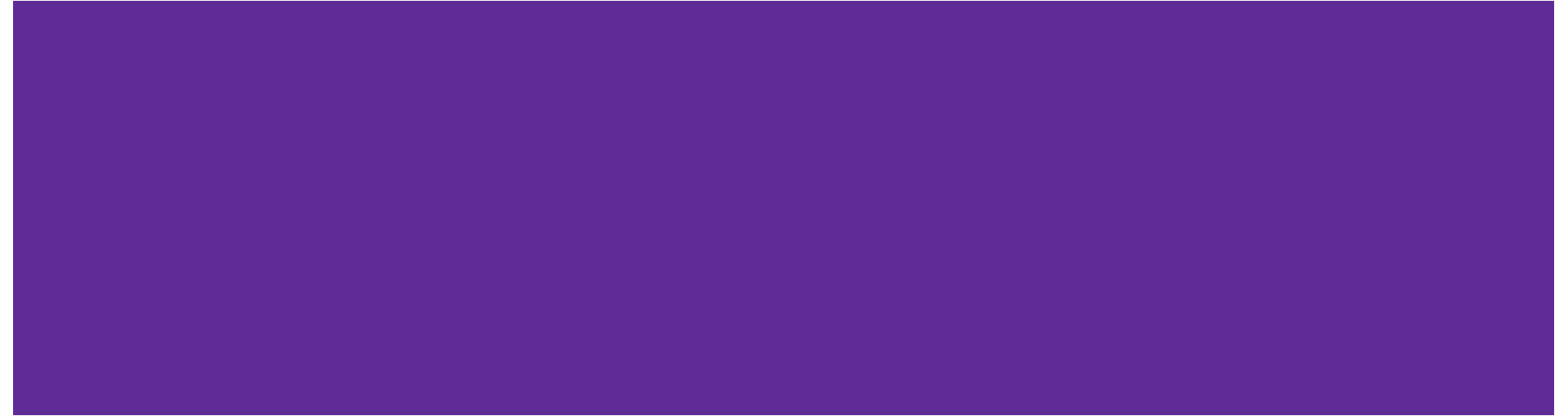


Unit-4

Advance in C#



- Delegates
 - Single Delegate
 - Multiple Delegate
- events
- Lambda Expression
 - Func
 - Action
 - Predicate
- LINQ
 - Method Syntax
 - Query Syntax
- Exception Handling
- Working with Database
 - ADO .NET
 - SqlDataReader
 - SqlDataAdapter
- Writing Web Application
 - ASP .NET
 - Web Forms

Delegate

A powerful feature that holds the reference to other methods which matches the return type & parameter

Allow method to be added or passed as a parameter

Can hold normal function/instance method/static method

act as a function pointer in c++ but type

They are the foundation for events and callback functions in C#.

Syntax:

```
access_modifier delegate return_type DelegateName (parameter_list);
```

Delegate Definition

```
delegate int DelegateName(int x, int y);
```

Method Definition

```
public int MethodName(int x, int y);  
static int MethodName(int x, int y);
```

Delegate Instantiation

```
DelegateName del = new DelegateName(MethodName);  
DelegateName del = MethodName; // Method group conversion
```

Delegate Invoke

```
int result = del(5, 3);
```

Multicast Delegate

```
del += AnotherMethod;  
del -= MethodName;
```

Types of Delegates:-

1. Single Cast Delegate

- a. Holds a single method reference.
- b. Used to call one method with a specific signature.

2. Multicast delegate

- a. Delegate holding reference of multiple methods
- b. Calls all methods in the invocation list sequentially.

3. Anonymous delegates

- a. provide a way to define an inline delegate
- b. Doesn't need to create separate method

4. Event Delegates

5. Func/Predicate/Action Delegates

Types of Delegates:-

1. Single Cast Delegate

- a. Holds a single method reference.
- b. Used to call one method with a specific signature.

2. Multicast delegate

- a. Delegate holding reference of multiple methods
- b. Calls all methods in the invocation list sequentially.

3. Anonymous delegates

- a. provide a way to define an inline delegate
- b. Doesn't need to create separate method

4. Event Delegates

5. Func/Predicate/Action Delegates

Write a C# program using delegates to perform arithmetic operations.

- Delegates:
 - ArithmeticOperation
 - Parameter 2, type int
 - Return type int
- 3 static Methods
 - Add, Subtract, Multiply
- Add all this method in a delegate with multicast delegates
- Invoke the delegate

Write a C# program using multicast delegates to send the same notification message through multiple channels: Email, SMS, and Push Notification.

- Delegates:
 - NotifyUser
 - Parameter 1, type string
 - Return type void
- 3 static Methods in a class called Notifier
 - SendEmail, SendSMS, SendPushNotification
- Add all this method in a delegate with multicast delegates
- Invoke the delegate

Event Member/ event keyword

Events are special member of a class used for triggering notification using delegates.

provide a restriction that limits who can invoke the event delegate.

- Other classes can only subscribe (+=) or unsubscribe (-=).
- Only the class that declares the event can invoke it.

Syntax:

```
delegate void CustomEventDelegate (string message);
```

```
public event CustomEventDelegate EventName;
```

Event Handler

A built-in delegate type used to define standard events.

It's a generic type delegate

Syntax:

```
public event EventHandler MyEvent;  
public delegate void EventHandler(object sender, EventArgs e);
```

EventArgs

In .NET event pattern, A base class for passing data with events.

Class Task

Write a program where a **YouTube Channel** can notify its **Subscribers** about general announcements.

Class YoutubeChannel:

- Member Field[channelName]
- Member Event[SubscriberNotification, delegate= 1 parameter; return void]
- Member Method
 - NotifySubscribers(string message) [private, void]
 - SendAnnouncement[public,void]

Class Subscriber

- Member Field [personName]
- Member method[That can be added as reference to event Delegate in YoutubeChannel Class]

Anonymous Method/Function

A method/function without a proper name and definition but has the implementation.

In C# 2.0, Anonymous delegate are used to write/present an inline/unnamed function

Later in C# 3.0, Lambda Expression are modern syntax for writing anonymous function.

```
(string name) {  
    Console.WriteLine($"Hello {name} !");  
}
```

In C# 2.0[]

```
Action greet = delegate(string name) {  
    Console.WriteLine($"Hello {name} !");  
};  
greet("Sagar");
```

In C# 3.0[]

```
Action greet = (name) => Console.WriteLine($"Hello {name}!");  
greet("Sagar");
```

Lambda Expression

is a concise way to represent an anonymous/inline function using the `=>` operator.

Anonymous Delegate Types:-

- **Func<Tinput,Toutput>**

- Take multiple input value and return one.
- Can take upto 16 input values.

```
Func<int, int, int> add = (a, b) => a + b;  
Console.WriteLine(add(5, 3)); // Output: 8
```

- **Action<Tinput>**

- Can Take multiple input value and return void[No Return]
- Can take upto 16 input values.

```
Action<string> printMessage = message => Console.WriteLine(message);  
printMessage("Hello, World!"); // Output: Hello, World!
```

- **Predicate<Tinput>**

- Take multiple value and always return boolean.
- Can take upto 16 input values.

```
// Predicate<int> checks if an integer is greater than 10
Predicate<int> isGreaterThanTen = num => num > 10;
Console.WriteLine(isGreaterThanTen(15)); // Output: True
Console.WriteLine(isGreaterThanTen(5));  // Output: False
```

- **Expression<Tinput>**

Class Task

1. Create an `Action<string>` that writes a greeting message to the console.
2. Create an `Action<List<string>>` that prints each element of a list on a new line.
3. Create an `Action<double>` that logs the square of the number.
4. Create a `Func<double,double, double>` that calculates the area of a rectangle from its height & width.
5. Create a `Func<int, bool>` that checks if a number is a even or not.
6. Create a `Func<string, string>` that returns the reversed version of a given string.
7. Create a `Predicate<char>` that checks if the character is an uppercase letter.
8. Create a `Predicate<int>` that checks if the number is a multiple of 5.
9. Create a `Predicate<List<int>>` that returns true if the list contains any negative numbers.

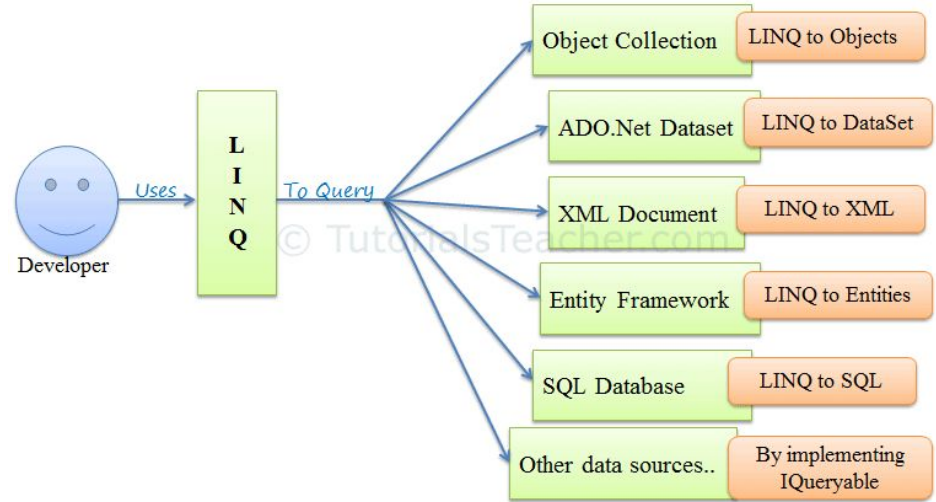
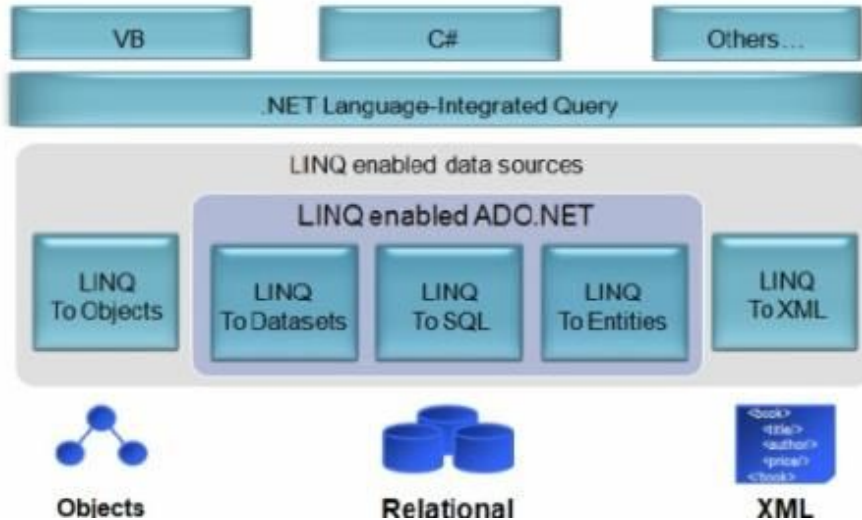
Use both Lambda Expression & Inline Delegate

LINQ(Language Integrated Query):

- It is a feature in C# (and other .NET languages) that provides a consistent way to query various types of data sources (arrays, collections, databases, XML, etc.).
- LINQ integrates query capabilities directly into the C# language.

LINQ(Language Integrated Query):

LINQ Overview



LINQ(Language Integrated Query):

LINQ Providers

- LINQ to Objects: Queries in-memory collections (e.g., arrays, lists).
- LINQ to XML: Queries XML documents.
- LINQ to SQL: Queries relational databases.
- LINQ to Entities: Queries Entity Framework databases.
- LINQ to DataSet: Queries data in a DataSet.

LINQ Syntax

Query Syntax (similar to SQL):

```
var query = from item in collection  
where item.Property == value  
select item;
```

Method Syntax (using extension methods):

```
var query = collection.Where(item => item.Property == value).Select(item  
=> item);
```

Query Syntax List

- **From Clause**
 - Specifies the data source and introduces the range variable.
 - from p in people
- **Where Clause**
 - Filters the sequence based on a condition.
 - where p.Age >= 18
- **Select Clause**
 - Projects the result into a new form.
 - select p.Name
- **OrderBy Clause**
 - Sorts the elements in ascending/descending order.
 - order by p.Age
- **Group By Clause**
 - Groups elements based on a key.
 - group p by p.Age

Method Syntax List

Query Operators

- Where
- OfType
- Cast

Projection Operators

- Select
- SelectMany

Sorting Operators

- OrderBy
- OrderByDescending

Aggregation Operators

- Count
- Sum
- Min
- Average

Class Task

Product[Name(string),Price(double),Category(string), InActive(boolean),stock(int)]

1. Given a list of products, how would you retrieve only the ones that are active?
2. How can you sort a list of products by their price in descending order?
3. How would you extract a list of distinct Category values from a list of Product objects?
4. How can you check if any product in a list has Stock equal to 0?
5. From a list of Product objects, how would you create a new list containing only the Name and Price properties?

Use both Array & Collection [using both Query Syntax & Method Syntax]

Class Task

Employee[Name(string), Salary(double), Department(string), IsResigned(boolean),
Experience(int)]

1. Given a list of employees, how would you retrieve only the ones who are currently working (not resigned)?
2. How can you sort a list of employees by their salary in descending order?
3. How would you extract a list of distinct Department values from a list of Employee objects?
4. How can you check if any employee in the list has zero years of experience?
5. From a list of Employee objects, how would you create a new list containing only the Name and Salary properties?

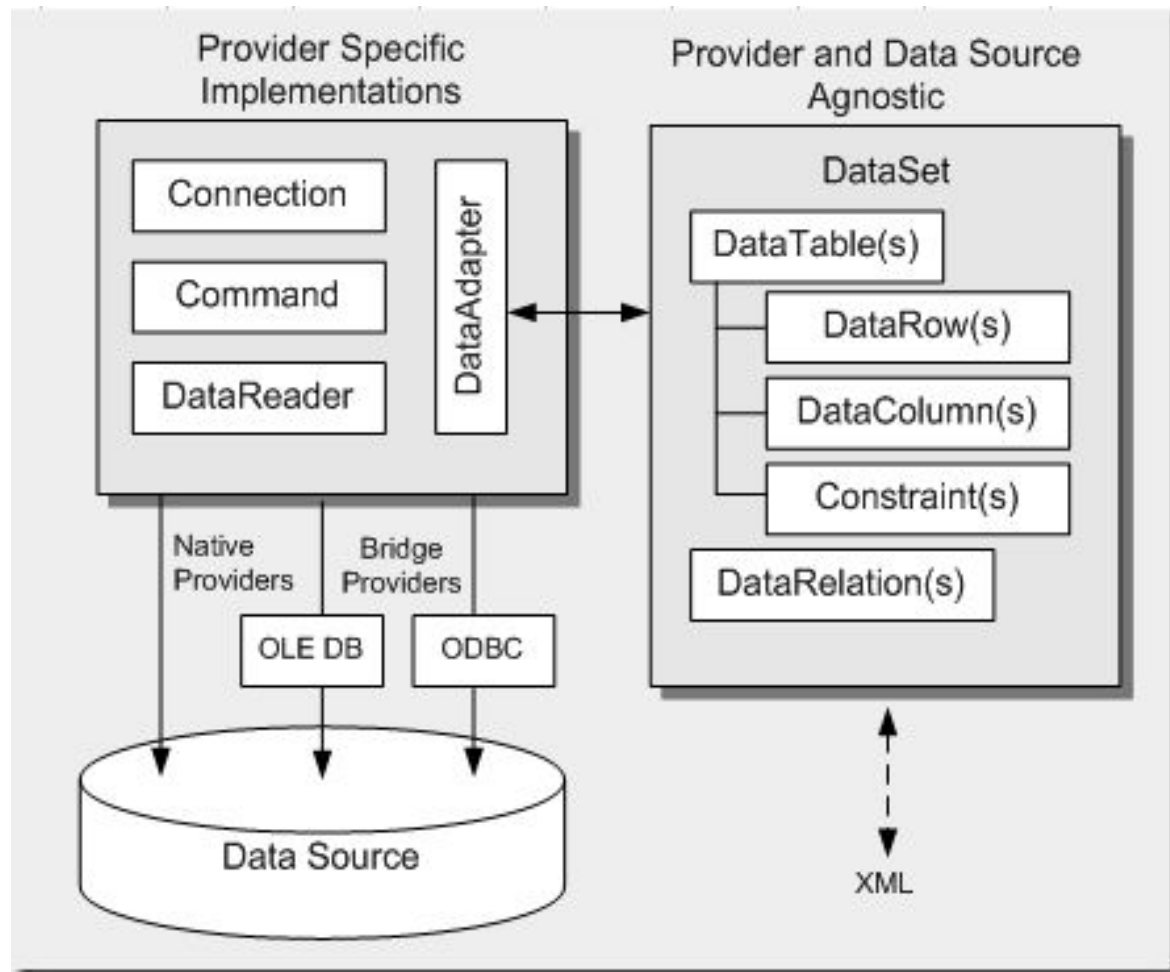
Use both Array & Collection [using both Query Syntax & Method Syntax]

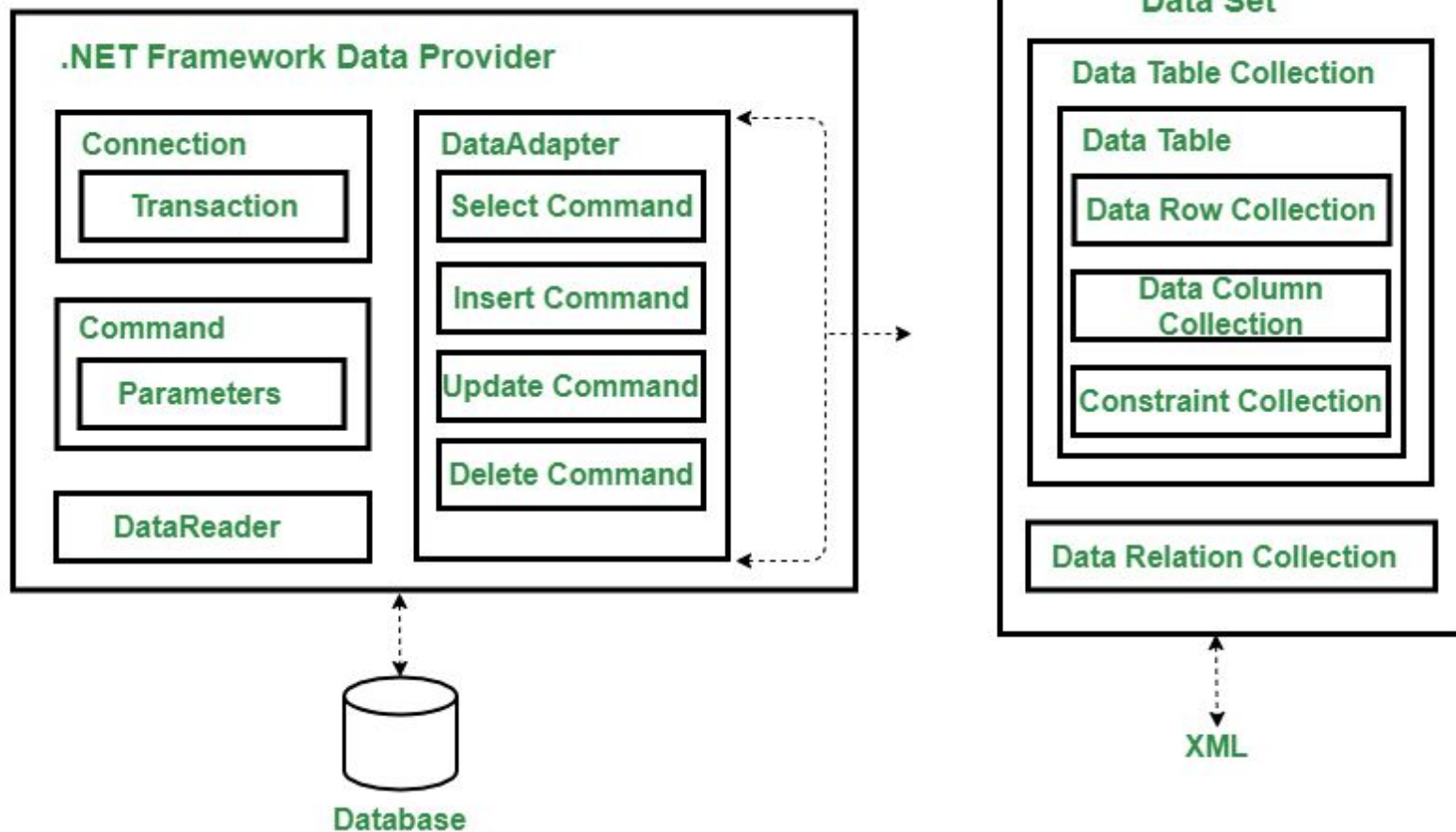
ADO .NET Overview:

Access Data Object(ADO) and ADO .NET

ADO.NET is a data access framework that provides a set of objects for managing database connections, executing commands, and retrieving or updating data

It can be used to interact with both connected (real-time connection) and disconnected (data retrieved and manipulated offline) data models.





Key Components

- Data Provider
 - Connection (Connects to a data source)
 - Command (Create & Executes SQL commands)
 - Data Reader (Fast, forward-only data reader)
 - Data Adapter (Bridges DataSet and data source)

Npgsql , MySql.Data, SqlServer

- DataSet
 - DataTable
 - DataRow
 - DataColumn

Data Provider

designed to interact with a specific type of data source (like SQL Server, Oracle, MySQL, etc.)

low-level infrastructure of ADO.NET responsible for establishing a connection to a data source, executing commands, and retrieving results.

It provides a way to interact with different types of databases in a consistent manner, regardless of the underlying database technology.

Data Provider Components:-

- Connection
 - Manages the connection between application & data base.
 - Uses a connection string to identify the data source, credentials, and database options.

Connection Providers:

- SqlConnection (for SQL Server)
- MySqlConnection (for MySQL data sources)
- NpgsqlConnection (for Postgres data sources)

```
string connStr = "Data Source=localhost;Initial Catalog=MyDB;Integrated Security=True";  
using (SqlConnection conn = new SqlConnection(connStr))  
{  
    conn.Open();  
    // .....  
}
```

- Command

- Execute SQL statement against a database
- It works together with a SqlConnection to perform actions in the database.

Command Providers:

- SqlCommand (for SQL Server)
- MySqlCommand (for MySQL data sources)
- NpgsqlCommand (for Postgres data sources)

```
using (SqlConnection conn = new SqlConnection(connStr))
{
    string query = "SELECT COUNT(*) FROM Users" ;
    SqlCommand cmd = new SqlCommand(query, conn);
    conn.Open();
    int userCount = (int)cmd.ExecuteScalar();

    Console.WriteLine("Total Users: " + userCount);
}
```

Command Methods

- `ExecuteReader()`
 - Execute a Command that returns multiple rows[Select]
 - Returns a `SqlDataReader` to read rows
- `ExecuteNonQuery()`
 - Executes a command that doesn't return rows (e.g., INSERT, UPDATE, DELETE)
 - Returns no of Rows Affected.
- `ExecuteScalar()`
 - Returns the first column of the first row (single value)

Data Reader

is a forward-only, read-only stream of rows from a SQL Server database

Features:

Connected: Requires an open SqlConnection.

Forward-only: Reads one row at a time; can't go backward.

Read-only: Cannot modify the data.

Efficient: Uses minimal memory compared to DataSet or DataTable.

```
Using cn As New SqlConnection("conn_string"), _  
    cmd As New SqlCommand("Select * from users", cn)  
  
    cn.Open()  
    Using rdr As SqlDataReader = cmd.ExecuteReader()  
        While rdr.Read()  
            '#' ...  
        End While  
    End Using  
End Using
```

Method:

Read(): Advances to the next row

ExecuteQuery() Syntax:

```
Using cn As New SqlConnection("conn_string"), _  
    cmd As New SqlCommand("INSERT INTO Users (Username, Age) VALUES (@Username, @Age)", cn)  
  
    //Add parameters  
    cmd.Parameters.AddWithValue("@Username", "john_doe")  
    cmd.Parameters.AddWithValue("@Age", 30)  
  
    cn.Open()  
    rowsAffected As Integer = cmd.ExecuteNonQuery()  
    Console.WriteLine("Rows inserted: " & rowsAffected)  
End Using
```

Data Adapter

acts as a bridge between a DataSet and a data source.

It is used to fetch data from a database into a DataSet and push changes back to the database.

- Fill a DataSet or DataTable with data from a database
- Send updates from DataSet/DataTable back to the database
- Works with commands: SELECT, INSERT, UPDATE, DELETE

Methods:

Fill(DataSet): Fills a DataSet or DataTable with query results

Update(DataSet): Updates the database with changes in DataSet

Properties:

SelectCommand:

InsertCommand:

UpdateCommand:

DeleteCommand:

DataSet

is an in-memory, disconnected, offline representation of data that provides a way to work with data retrieved from a database.

A DataSet can work offline, meaning you can retrieve, manipulate, and update data without a constant connection to the database.

DataAdapter Syntax(Select & Fill):

```
Using cn As New SqlConnection( "conn_string"), _  
    adapter As New SqlDataAdapter( "SELECT * FROM users", cn)  
  
Dim ds As New DataSet()  
adapter.Fill(ds, "users")  
  
For Each row In ds.Tables( "users").Rows  
    '# Access data using: row( "column_name")  
Next  
End Using
```

DataAdapter Syntax(Insert & Update):

```
// INSERT a new row
    DataRow newRow = ds.Tables["users"].NewRow();
    newRow["Username"] = "new_user";
    newRow["Age"] = 25;
    ds.Tables["users"].Rows.Add(newRow);

// UPDATE an existing row (example: update first row)
    if (ds.Tables["users"].Rows.Count > 0)
    {
        ds.Tables["users"].Rows[0]["Age"] = 30;
    }

// Auto-generate INSERT and UPDATE commands
    SqlCommandBuilder builder = new SqlCommandBuilder(adapter);
    adapter.InsertCommand = builder.GetInsertCommand();
    adapter.UpdateCommand = builder.GetUpdateCommand();

// Apply changes to the database
    adapter.Update(ds, "users");
```

Class Task

1. Write code to retrieve all products with `InActive = 0` using `SqlCommand.ExecuteReader()`. Display the product Name and Price in the console.
2. Write code to insert a new product into the Product table using `SqlCommand.ExecuteNonQuery()`. The product details (Name, Price, Category, InActive, Stock) should be passed as parameters.
3. Use `SqlDataAdapter.Fill()` to load all products into a `DataSet`. Loop through the rows and display the Name, Category, and Stock of each product.
4. Load all products into a `DataSet` using `SqlDataAdapter.Fill()`. Update the stock of a specific product (e.g., increase stock by 10 for a given product name) in the `DataSet`, then save changes back to the database using `SqlDataAdapter.Update()`.

Class Task

1. Write code to retrieve all employees where `IsResigned = 0` using `SqlCommand.ExecuteReader()`. Display employee Name and Salary in the console.
2. Write code to insert a new employee into the Employee table using `SqlCommand.ExecuteNonQuery()`. Pass employee details (Name, Salary, Department, `IsResigned`, Experience) as parameters.
3. Use `SqlDataAdapter.Fill()` to load all employees into a `DataSet`. Loop through the rows and display the Name, Department, and Experience of each employee.
4. Load all employees into a `DataSet` using `SqlDataAdapter.Fill()`. Increase the Experience of a specific employee (e.g., by 1 year for a given Name) in the `DataSet`. Then save changes back to the database using `SqlDataAdapter.Update()`.

Summary

1. Using Command

- a. For Select
 - i. Connection -> Command -> executeReader -> .Read()
- b. For Insert/Update/Delete
 - i. Connection -> Command -> executeNonQuery() -> noOfRows Affected

2. Using DataAdapter

- a. For Select
 - i. Connection -> DataAdapter-> Fill(dataSet)-> readDataSet
- b. For Insert/Update/Delete
 - i. Connection-> DataAdapter->Fill(dataSet) -> dataSet operation -> sqlCommandBuilder -> update(datasets)

Questions [Assignment 4]:-

1. Explain the difference between connected and disconnected architecture.
2. What is the role of Connection, Command, DataReader, and DataAdapter?
3. What is the difference between ExecuteReader, ExecuteScalar, and ExecuteNonQuery?
4. What is a DataSet and how is it different from a DataReader?
5. How do you update the database using DataAdapter?

ASP .NET

part of the .NET framework, and it's used for building dynamic and enterprise level web applications and services.

built on the CLR(Common Language Runtime) which allows the programmers to execute its code using any .NET language(C#, VB etc.)

History and Advancement of ASP

ASP (Classic):

- Supports server-side scripting using VBScript and JavaScript.
- Uses sessions and application objects for state management.

ASP.NET (1.0 - 4.0)

- Introduces a component-based model with server controls and an event-driven programming model.[Web Forms]
- Uses View State and other mechanisms for managing state across postbacks.

.

ASP.NET MVC (Model-View-Controller):

- Clearly separates the application into Model, View, and Controller components.[Seperation of concern]
- Provides a clean URL routing system to manage URLs in a more meaningful way.

ASP.NET Web API:

- Facilitates the creation of HTTP services that can be accessed from a variety of clients including browsers and mobile devices.

ASP.NET Core:

- Runs on Windows, Linux, and macOS.
- ASP .NET Core MVC; ASP .NET Core API

Webforms

ASP.NET Web Forms is one of the original frameworks for building dynamic web applications in the .NET ecosystem

Introduces a component-based model with server controls and an event-driven programming model.

FEATURES OF WEBFORM --->

- Component Based Model
- Server Control
- Event Driven Programming
- State Management
- Master Page
- Security

Component Based Model

It relies on reusable, pre-built, self-contained server-side components (controls) to build the user interface (UI).

Data Collecting Control: <asp:TextBox>, <asp:Button>, <asp:CheckBox>, <asp:ImageButton>

Viewing Control: <asp:GridView>, <asp:DataList>, <asp:ListView>, <asp:Repeater>

Validation Control: <asp:RequiredFieldValidator>, <asp:RangeValidator> etc.

Advantages:

- Encapsulation of Functionality
- Reusability
- Abstraction

Server Control

are special components that run on the server and generate HTML markup that is sent to the client browser.

These controls execute their logic on the server. When a page is requested by the client, the server processes these controls, generates the appropriate HTML, and sends the result back to the client browser.

Server-side controls support event-driven programming. For example, clicking a button triggers a Click event, which can be handled with a server-side method

Event Driven Programming

The flow of the application is determine based on user interaction/change in action[event] and selections trigger events that execute corresponding event handler methods on the server.

State Management

ViewState is a client-side mechanism that stores the state of controls between postbacks.

This means user input is preserved between page loads unless explicitly disabled.

Scope:- Per Page

Stored:- Client Side

Accessible:- Same Page

SessionState is a server-side mechanism that stores user-specific data across multiple pages for the duration of the user's session.

Scope:- Per User

Stored:- Server Side

Accessible:- Across Page

ApplicationState is a global storage accessible to all users and sessions across the entire application. Stored on the server.

Scope:- Global

Stored:- Server Side

Accessible:- Across Sessions

ControlState

Hidden Value

Cookies

Query String

Master Page

They are basic templates for creating a consistent layout for multiple pages in a web application.

```
<%@ Master Language="C#" AutoEventWireup="true" CodeBehind="Site.master.cs"
Inherits="ASPNETWebform.Site" %>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head runat="server">
```

```
    <title> </title>
```

```
</head>
```

```
<body>
```

```
    <div>
```

```
        <asp:ContentPlaceHolder ID="MainContent" runat="server" />
```

```
    </div>
```

```
</body>
```

```
</html>
```

LoginPage

```
<%@ Page Title="" Language="C#" MasterPageFile="~/Site.Master" AutoEventWireup="true"  
CodeBehind="AddProduct.aspx.cs" Inherits="ASPNETWebform.AddProduct" %>
```

```
<asp:Content ID="Content2" ContentPlaceHolderID="MainContent" runat="server">  
    <asp:Label ID="UsernameLabel" runat="server" Text="UserName"></asp:Label>  
    <asp:TextBox ID="Username" runat="server"></asp:TextBox>
```

```
<br />
```

```
<asp:Label ID="PasswordLabel" runat="server" Text="Password"></asp:Label>  
<asp:TextBox ID="Password" runat="server"></asp:TextBox>
```

```
<br />
```

```
<asp:Button ID="ButtonSubmit" runat="server" Text="Login" />
```

```
</asp:Content>
```

Repeater

- Introduced in ASP .NET 3.5 or later
- No built-in layout or styling – you define everything manually.
- Does not support paging, sorting, or editing by default.
- Ideal for full custom layout control.

Datalist

- Introduced in ASP .NET 2.0
- Lightweight and faster for displaying simple multi column lists
- Supports basic paging and selection, less feature-rich
- Less flexible; relies on RepeatDirection and RepeatColumns

ListView

- Introduced in ASP .NET 3.5 or later
- Complex, fully customizable layouts with flexible templates,
- Supports paging, editing, inserting, deleting with templating and DataPager
- Easier to customize with CSS & HTML due to full template control

Use Repeater: When you need full HTML control and best performance.

Use DataList: When you want simple multi-column layouts and easy layout setup.

Use ListView: When you need full features (paging, sorting, editing) with full layout flexibility.